Bölüm 6 Diziler

Açılış Problemi

Yüz sayı okuyun, ortalamasını hesa playın ve kaç sayının olduğunu bulun.

ortalamanın üzerinde.

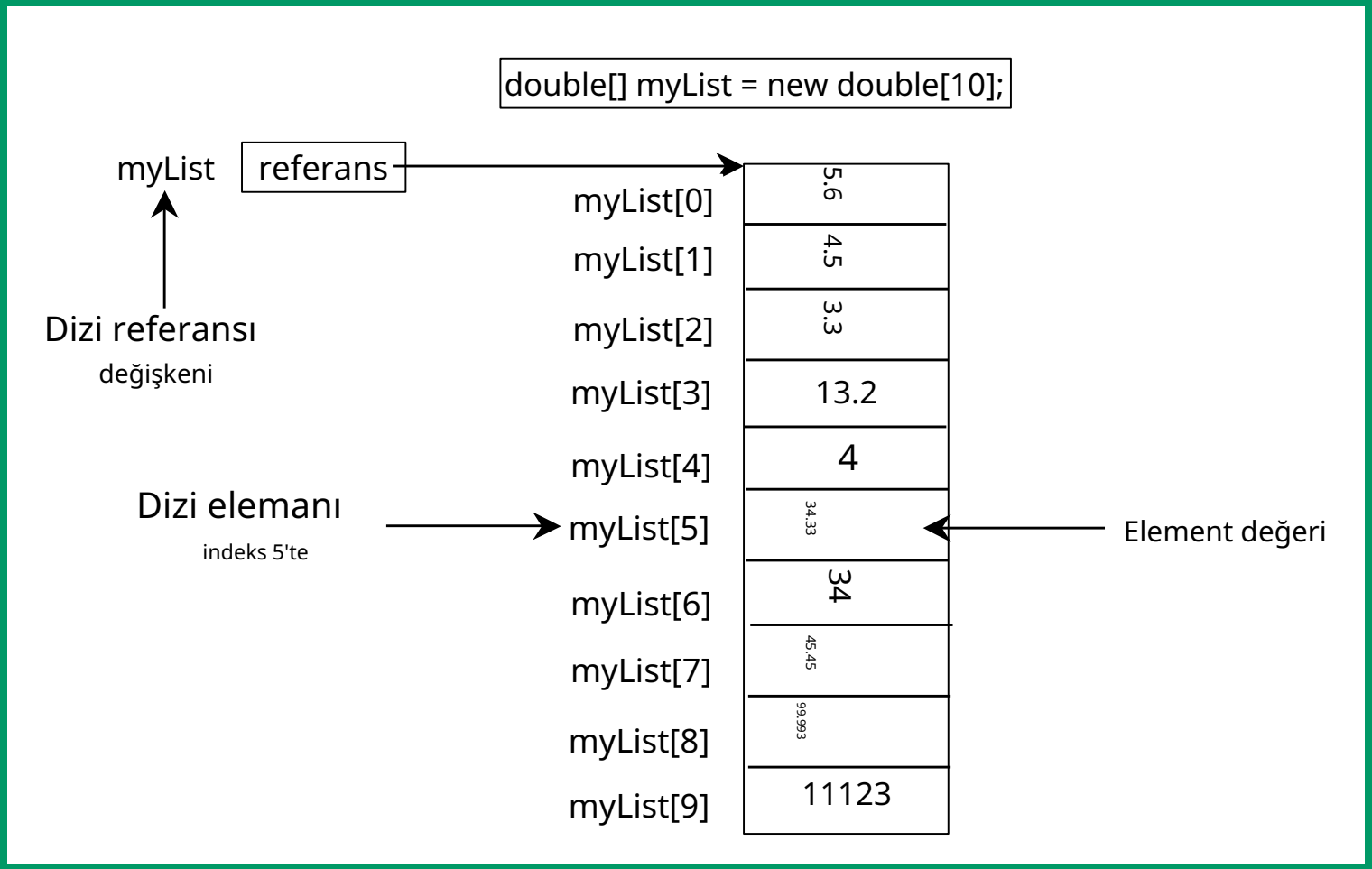
Çözüm

AnalyzeNumbers

Koş

Dizileri Tanıtma

Dizi, bir koleksiyonu temsil eden bir veri yapısıdır.
aynı türde veriler.



Dizi Değişkenlerini Tanımlama

- veri türü[] diziReferansDeğişkeni;

  Örnek:

```
double[] myList;
```

- veri türü arrayRefVar[]; // Bu stil izin verilir, ancak tercih edilmez

  Örnek:

```
double myList[];
```

Diziler Oluşturma

arrayRefVar = new veri türü[arraySize];

Örnek:

```
myList = new double[10];
```

`myList[0]` dizideki ilk öğeyi referans alır.

`myList[9]` dizideki son öğeyi referans alır.

Tanımlama ve Oluşturma Tek Adımda

- veriTipi[] diziRefDeğişkeni = new datatype[diziBoyutu];

```
double[] myList = new double[10];
```

- veriTipi diziRefDeğişkeni[] = new veriTipi[diziBoyutu];

```
double myList[] = new double[10];
```

Bir Dizinin Uzunluğu

Bir dizi oluşturulduğunda, boyutu sabittir. Değiştirilemez. değiştirildi. Boyutunu bulmak için

arrayName.length

Örneğin,

myList.length 10 döner

Varsayılan Değerler

Bir dizi oluşturulduğunda, elemanları varsayılan değeri alır

0 sayısal ilkel veri türleri için,

char türleri için '\u0000' ve

boolean türleri için yanlış.

İndeksli Değişkenler

Dizi elemanlarına indeks aracılığıyla erişilir. Dizi indeksleri 0 tabanlıdır, yani 0'dan başlar ve arrayRefVar.length-1'e kadar gider. Şekil 6.1'deki örnekte, myList on adet double değeri tutar ve indeksler 0'dan 9'a kadardır.

Dizideki her bir eleman aşağıdaki sözdizimi ile temsil edilir, bu sözdizimi indeksli değişken olarak bilinir:

```
diziAdı[indeks];
```

İndeksli Değişkenlerin Kullanımı

Bir dizi oluşturulduktan sonra, indeksli bir değişken normal bir değişken gibi kullanılabilir. Örneğin, aşağıdaki kod myList[0] ve myLis t[1] değerlerini myList[2]'ye ekler.

```
myList[2] = myList[0] + myList[1];
```

Dizi Başlatıcıları

- Bir adımda beyan etme, oluşturma, başlatma:

```
double[] myList = {1.9, 2.9, 3.4, 3.5};
```

Bu kısayol sözdizimi bir

ifadede olmalıdır.

Beyan etme, oluşturma, başlatma Kısayol Notasyonu Kullanarak

```
double[] myList = {1.9, 2.9, 3.4, 3.5};
```

Bu kısayol notasyonu, aşağıdaki ifad elerle eşdeğerdir:

```
double[] myList = new double[4];
myList[0] = 1.9;
myList[1] = 2.9;
myList[2] = 3.4;
myList[3] = 3.5;
```

DİKKAT

Kısayol notasyonunu kullanarak, siz diziyi tek bir ifadede beyan etmeli, o luşturmalı ve başlatmalısınız.

Parçalamak bir sözdizimi hatasına neden olur.

hata. Örneğin, aşağıdakiler şunlardır yanlış:

```
double[] myList;

myList = {1.9, 2.9, 3.4, 3.5};
```

animasyon

Trace Program with Arrays

Değişken dizisi values'ı tanımlayın, bir dizi oluşturun ve referansını values'a atayın

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

Dizi oluşturulduktan sonra

0	0
1	0
2	0
3	0
4	0

animasyon

Trace Program with Arrays

i 1 olur

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

Dizi oluşturulduktan sonra

0	0
1	0
2	0
3	0
4	0

animasyon

Trace Program with Arrays

i (=1) 5'ten küçüktür

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

Dizi oluşturulduktan sonra

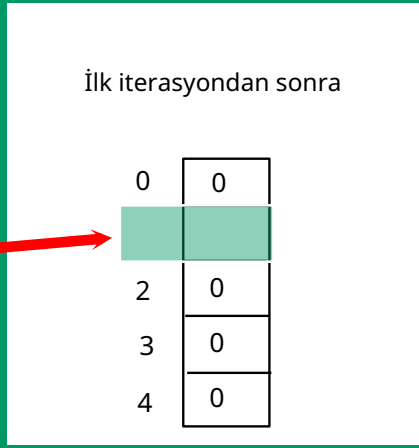
0	0
1	0
2	0
3	0
4	0

animasyon

Trace Program with Arrays

Bu satır çalıştırıldıktan sonra, value[1] 1'dir

```
public class Test {
  public static void main(Strin[]  ags) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```



animasyon

Trace Program with Arrays

i++'dan sonra, i 2 olur

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

İlk iterasyondan sonra

0	0
1	1
2	0
3	0
4	0

animasyon

Trace Program with Arrays

```
public class Test {
  public static void main(String[]
      args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] +
      values[4];
  }
}
```

i (= 2) 5'ten küçüktür

İlk iterasyondan sonra

0	0
1	1
2	0
3	0
4	0

animasyon

Trace Program with Arrays

Bu satır çalıştırıldıktan sonra,
values[2] 3'tür (2 + 1)

```
public class Test {
  public static void main(Strin[] ...gs) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

İkinci iterasyondan sonra

0	0
1	1
2	3
3	0
4	0

animasyon

Trace Program with Arrays

Bu noktadan sonra, i 3 olur.

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

İkinci iterasyondan sonra

0	0
1	1
2	3
3	0
4	0

animasyon

Trace Program with Arrays

i (=3) hala 5'ten küçüktür.

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

İkinci iterasyondan sonra

0	0
1	1
2	3
3	0
4	0

animasyon

Trace Program with Arrays

Bu satırdan sonra, values[3] 6 olur (3 + 3)

```
public class Test {
  public static void main(String[] arg
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

Üçüncü yinelemeden sonra

0	0
1	1
2	3
3	6
4	0

animasyon

Trace Program with Arrays

Bu noktadan sonra, i 4 olur.

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

Üçüncü yinelemeden sonra

0	0
1	1
2	3
3	6
4	0

animasyon

Trace Program with Arrays

i (=4) hala 5'ten küçüktür

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

Üçüncü yinelemeden sonra

0	0
1	1
2	3
3	6
4	0

animasyon

Trace Program with Arrays

Bundan sonra, values[4] 10 (4 + 6) olur

```
public class Test {
  public static void main(Strin[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

Dördüncü yinelemeden sonra

0	0
1	1
2	3
3	6

animasyon

Trace Program with Arrays

i++'den sonra, i 5 olur

```
public class Test {
  public static void main(String[] arg
        {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

Dördüncü yinelemeden sonra

0	0
1	1
2	3
3	6
4	10

animasyon

Trace Program with Arrays

i (=5) < 5 yanlıştır. Döngüden çık.

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

Dördüncü yinelemeden sonra

0	0
1	1
2	3
3	6
4	10

animasyon

Trace Program with Arrays

Bu satırdan sonra, values[0] 11'dir (1 + 10)

```
public class Test {
  public static void m[illegible]ring[] args) {
    int[] values = ne[illegible][5];
    for (int i = 1; i [illegible], i++) {
      values[i] = i[illegible] values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

0	11
1	1
2	3
3	6
4	10

Dizileri İşleme

Metindeki örneklere bakın.

1. (Giriş değerleri ile dizileri başlatma)
2. (Rastgele değerlerle dizileri başlatma)
3. (Dizileri yazdırma)
4. (Tüm elemanları toplama)
5. (En büyük elemanı bulma)
6. (En büyük elemanın en küçük indeksini bulma)
7. (*Shifting elements*)

Dizileri giriş değerleriyle başlatma

```
java.util.Scanner input = new java.util.Scanner(System.in);
System.out.print("Lütfen " + myList.length + " değer girin: ");
for (int i = 0; i < myList.length; i++)
  myList[i] = input.nextDouble();
```

Dizileri rastgele değerlerle başlatma

```
for (int i = 0; i < myList.length; i++) {
  myList[i] = Math.random() * 100;
}
```

Dizileri yazdırma

```
for (int i = 0; i < myList.length; i++) {
  System.out.print(myList[i] + " ");
}
```

Tüm elemanları toplama

```
double total = 0;
for (int i = 0; i < myList.length; i++) {
  total += myList[i];
}
```

En büyük elemanı bulma

```
double max = myList[0];
for (int i = 1; i < myList.length; i++) {
  if (myList[i] > max)
      max = myList[i];
}
```

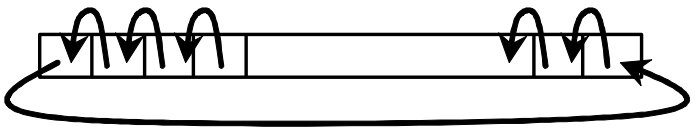
Elemanları Taşıma

```
double temp = myList[0];  // İlk elemanı koru

// Elemanları sola kaydır
for (int i = 1; i < myList.length; i++) {
    myList[i - 1] = myList[i];
}

// İlk elemanı son pozisyonu doldurmak için taşı
myList[myList.length - 1] = temp;
```

myList



Geliştirilmiş for Döngüsü (her biri için döngü)

JDK 1.5, bir diziyi indeks değişkeni kullanmadan sıralı bir şekilde dolaşmanıza olanak tanıyan yeni bir for döngüsü tanıttı. Örneğin, aşağıdaki kod myList dizisindeki tüm öğeleri görüntüler:

```
for (double value: myList)
  System.out.println(value);
```

Genel olarak, sözdizimi

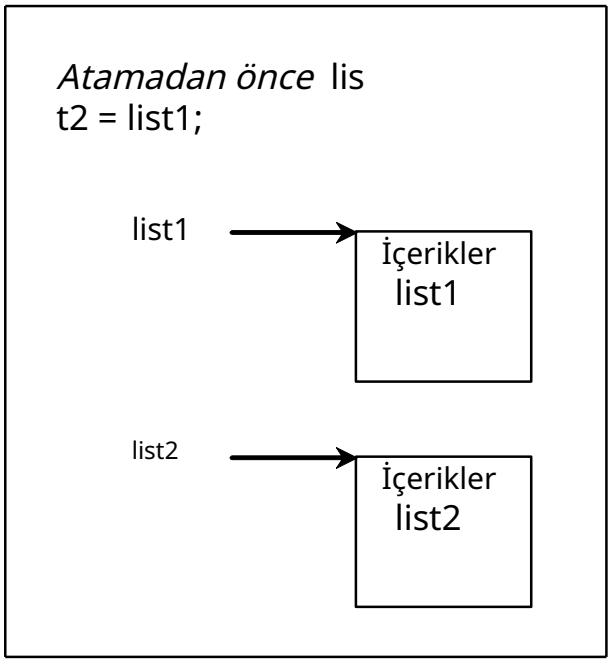
```
for (elementType value: arrayRefVar) {
  // Değeri işleyin
}
```

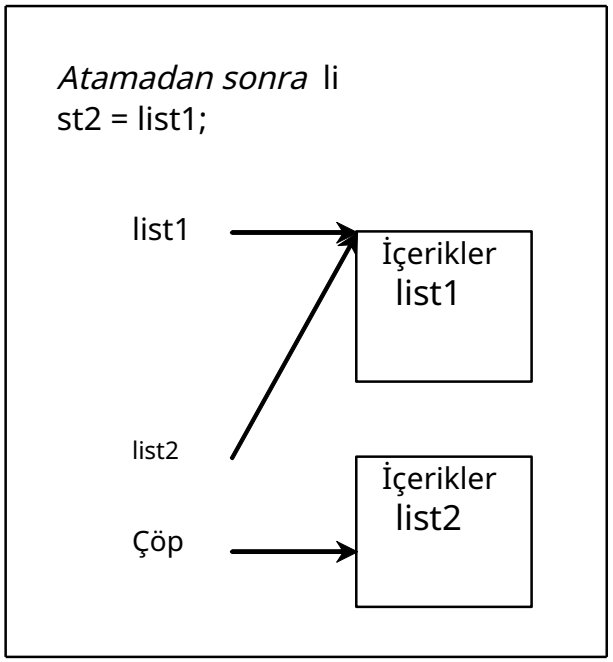
Array'i farklı bir sırayla gezmek veya array'deki elemanları değiştirmek istiyorsanız, hala bir indeks değişkeni kullanmanız gerekiyor.

Dizileri Kopyalama

Sıklıkla, bir programda, bir diziyi veya bir dizinin bir kısmını kopyalama nız gerekir. Bu gibi durumlarda atama ifadesini kullanmayı deneyebilirsiniz. (=), aşağıdaki gibi:

```
list2 = list1;
```





Dizileri Kopyalama

Bir döngü kullanarak:

int[] kaynakDizi = {2, 3, 1, 5, 10};

int[] hedefDizi = new
int[kaynakDizi.length];

```
for (int i = 0; i < sourceArrays.length; i++)
    targetArray[i] = sourceArray[i];
```

arraycopyUtility

```
arraycopy(sourceArray, src_pos,
  targetArray, tar_pos, length);
```

Örnek:

```
System.arraycopy(sourceArray, 0,
  targetArray, 0, sourceArray.length);
```

Dizileri Yöntemlere Geçirme

```
public static void printArray(int[] array) {
  for (int i = 0; i < array.length; i++) {
    System.out.print(array[i] + " ");
  }
}
```

Yöntemi çağırın

```
int[] liste = {3, 1, 2, 6, 4, 2};
printArray(liste);
```

Yöntemi çağır

```
printArray(new int[]{3, 1, 2, 6, 4, 2});
```

Anonim dizi

Anonim Dizi

İfade

```
printArray(new int[]{3, 1, 2, 6, 4, 2});
```

aşağıdaki sözdizimini kullanarak bir dizi oluşturur:

```
new dataType[]{literal0, literal1, ..., literalk};
```

Dizi için açık bir referans değişken yoktur. Bu dizi *anonim dizi* olarak adlandırılır.

Değerle Geçiş

Java, bir yönteme argümanları iletmek için *değer ile geçiş* kullanır. Değişkenlerin bir değerini iletmek arasında önemli farklar vardır. ilkel veri türleri ve dizileri geçirme.

Bir ilkel tür değeri için bir parametre, gerçek değer geçilir. Yerel parametrenin değerini değiştirmek metod, değişkenin dışındaki değerini etkilemez yöntem.

☞ Dizi türünde bir parametre için, parametrenin değeri diziye bir referans içerir; bu referans metoda gönderilir. Metodun gövdesi içinde meydana gelen diziye yapılan herhangi bir değişiklik, argüman olarak gönderilen orijinal diziyi etkileyecektir.

Basit Örnek

```
public class Test {
  public static void main(String[] args) {
    int x = 1; // x bir int değerini temsil eder
    int[] y = new int[10]; // y, int değerlerinin bir dizisini temsil eder

    m(x, y); // m'yi x ve y argümanlarıyla çağır

    System.out.println("x, " + x + " olarak tanımlandı.");
    System.out.println("y[0], " + y[0] + " olarak tanımlandı.");
  }

  public static void m(int number, int[] numbers) {
    number = 1001; // number'a yeni bir değer ata
    numbers[0] = 5555; // numbers[0]'a yeni bir değer ata
  }
}
```

Dizileri Argüman Olarak Geçirme

- Amaç:
  ilkel veri türü değişkenleri ile

  ve dizi değişkenleri arasındaki farkları göstermek.

TestPassArray

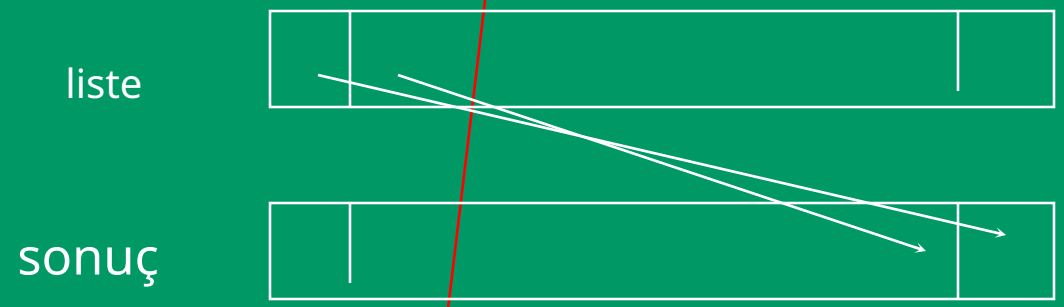
Koş

Bir Metottan Dizi Döndürme

```
public static int[] ters(int[] liste) {
    int[] sonuç = new int[liste.length];

    for (int i = 0, j = sonuç.length  - 1;
            i < liste.length; i++, j--) {
        sonuç[j] = liste[i];
    }

    return sonuç;
}
```




```
int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = ters(liste1);
```

animasyon

Trace the reverse Method

```
int[] list1 = {1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

Declare result and create array

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

liste

1	2	3	4	5	6

sonuç

0	0	0	0	0	0

animasyon

Trace the reverse Method, cont.

int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);

i = 0 ve j = 5

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < liste.length; i++, j--) {
    sonuç[j] = liste[i];
  }

  return result;
}
```

liste	1	2	3	4	5	6
sonuç	0	0	0	0	0	0

animasyon

Trace the reverse Method, cont.

int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);

i (= 0) 6'dan küçüktür

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    sonuç[j] = liste[i];
  }

  return result;
}
```

liste	1	2	3	4	5	6
sonuç	0	0	0	0	0	0

animasyon

Trace the reverse Method, cont.

```
int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);
```

```
public static int[] reverse(int[] liste) {
    int[] sonuç = new int[liste.length];

    for (int i = 0, j = sonuç.length - 1;
            i < liste.length; i++, j--) {
        sonuç[j] = liste[i];
    }

    return result;
}
```

i = 0 ve j = 5
list[0]'ı sonuç[5]'e ata

liste	1	2	3	4	5	6
sonuç	0	0	0	0	0	1

animasyon

Trace the reverse Method, cont.

```
int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);
```

```
public static int[] reverse(int[] liste) {
    int[] sonuç = new int[liste.length];

    for (int i = 0, j = sonuç.length - 1;
            i < liste.length; i++, j--) {
        sonuç[j] = liste[i];
    }

    return result;
}
```

Sonrasında, i 1 olur ve j 4 olur

liste	1	2	3	4	5	6
sonuç	0	0	0	0	0	1

animasyon

Trace the reverse Method, cont.

int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

i (=1) 6'dan küçüktür

liste	1	2	3	4	5	6
sonuç	0	0	0	0	0	1

animasyon

Trace the reverse Method, cont.

```
int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);
```

```
public static int[] reverse(int[] liste) {
    int[] sonuç = new int[liste.length];

    for (int i = 0, j = sonuç.length - 1;
            i < liste.length; i++, j--) {
        sonuç[j] = liste[i];
    }

    return result;
}
```

i = 1 ve j = 4
list[1]'i result[4]'e ata

liste	1	2	3	4	5	6
sonuç	0	0	0	0	2	1

animasyon

Trace the reverse Method, cont.

```
int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);
```

```
public static int[] reverse(int[] liste) {
    int[] sonuç = new int[liste.length];

    for (int i = 0, j = sonuç.length                    1;
            i < liste.length; i++, j--) {
        sonuç[j] = liste[i];
    }

    return result;
}
```

Bundan sonra, i 2 olur ve j 3 olur

liste	1	2	3	4	5	6
sonuç	0	0	0	0	2	1

animasyon

Trace the reverse Method, cont.

int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length]

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    sonuç[j] = liste[i];
  }

  return result;
}
```

i (=2) hala 6'dan küçüktür

liste	1	2	3	4	5	6
sonuç	0	0	0	0	2	1

animasyon

Trace the reverse Method, cont.

```
int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);
```

```
public static int[] reverse(int[] liste) {
    int[] sonuç = new int[liste.length];

    for (int i = 0, j = sonuç.length - 1;
            i < liste.length; i++, j--) {
        sonuç[j] = liste[i];
    }

    return result;
}
```

i = 2 ve j = 3

list[i] değerini result[j] olarak ata

liste	1	2	3	4	5	6
sonuç	0	0	0	3	2	1

animasyon

Trace the reverse Method, cont.

```
int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);
```

```
public static int[] reverse(int[] liste) {
    int[] sonuç = new int[liste.length];

    for (int i = 0, j = sonuç.length                    1;
            i < liste.length; i++, j--) {
        sonuç[j] = liste[i];
    }

    return result;
}
```

Bundan sonra, i 3 olur ve j 2 olur

liste	1	2	3	4	5	6
sonuç	0	0	0	3	2	1

animasyon

Trace the reverse Method, cont.

int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);

i (=3) hala 6'dan küçüktür

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length]

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    sonuç[j] = liste[i];
  }

  return result;
}
```

liste	1	2	3	4	5	6
sonuç	0	0	0	3	2	1

animasyon

Trace the reverse Method, cont.

```
int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);
```

```
public static int[] reverse(int[] liste) {
    int[] sonuç = new int[liste.length];

    for (int i = 0, j = sonuç.length - 1;
            i < liste.length; i++, j--) {
        sonuç[j] = liste[i];
    }

    return result;
}
```

i = 3 ve j = 2

list[i] değerini result[j] olarak ata

liste	1	2	3	4	5	6
sonuç	0	0	4	3	2	1

animasyon

Trace the reverse Method, cont.

```
int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);
```

```
public static int[] reverse(int[] liste) {
    int[] sonuç = new int[liste.length];

    for (int i = 0, j = sonuç.length                    1;
            i < liste.length; i++, j--) {
        sonuç[j] = liste[i];
    }

    return result;
}
```

Bundan sonra, i 4 olur ve j 1 olur

liste	1	2	3	4	5	6
sonuç	0	0	4	3	2	1

animasyon

Trace the reverse Method, cont.

int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

i (=4) hala 6'dan küçük

liste	1	2	3	4	5	6

sonuç	0	0	4	3	2	1

animasyon

Trace the reverse Method, cont.

```
int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);
```

```
public static int[] reverse(int[] liste) {
    int[] sonuç = new int[liste.length];

    for (int i = 0, j = sonuç.length - 1;
            i < liste.length; i++, j--) {
        sonuç[j] = liste[i];
    }

    return result;
}
```

i = 4 ve j = 1
list[i]'yi result[j]'ye ata

liste	1	2	3	4	5	6
sonuç	0	5	4	3	2	1

animasyon

Trace the reverse Method, cont.

int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

Bundan sonra, i 5 olur ve j 0 olur

liste	1	2	3	4	5	6

sonuç	0	5	4	3	2	1

animasyon

Trace the reverse Method, cont.

int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);

i (=5) hala 6'dan küçük

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length]

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    sonuç[j] = liste[i];
  }

  return result;
}
```

liste	1	2	3	4	5	6

sonuç	0	5	4	3	2	1

animasyon

Trace the reverse Method, cont.

```
int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);
```

```
public static int[] reverse(int[] liste) {
    int[] sonuç = new int[liste.length];

    for (int i = 0, j = sonuç.length - 1;
            i < liste.length; i++, j--) {
        sonuç[j] = liste[i];
    }

    return result;
}
```

i = 5 ve j = 0

list[i] değerini result[j] olarak ata

liste	1	2	3	4	5	6
sonuç	6	5	4	3	2	1

animasyon

Trace the reverse Method, cont.

int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

Bundan sonra, i 6 olur ve j -1 olur

liste	1	2	3	4	5	6

sonuç	6	5	4	3	2	1

animasyon

Trace the reverse Method, cont.

int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length]

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    sonuç[j] = liste[i];
  }

  return result;
}
```

i (=6) < 6 yanlıştır. Bu yüzden döngüden çık

liste	1	2	3	4	5	6

sonuç	6	5	4	3	2	1

animasyon

Trace the reverse Method, cont.

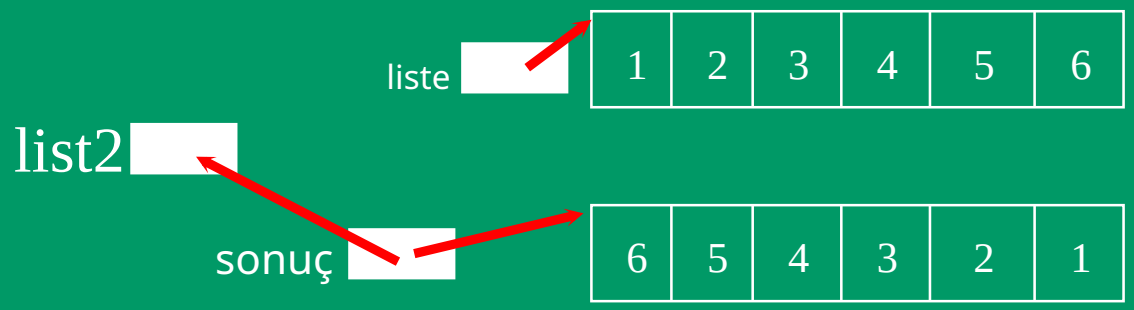
```
int[] liste1 = new int[]{1, 2, 3, 4, 5, 6};
int[] liste2 = reverse(liste1);
```

```
public static int[] reverse(int[] liste) {
    int[] sonuç = new int[liste.length];

    for (int i = 0, j = sonuç.length                    - 1;
            i < liste.length; i++, j--) {
        sonuç[j] = liste[i];
    }

    return result;
}
```

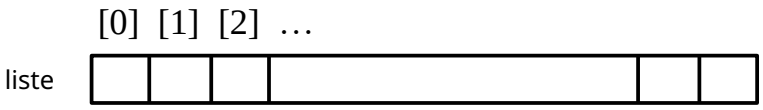
result değerini döndür



Dizileri Arama

Arama, bir dizide belirli bir öğeyi bulma sürecidir; örneğin, belirli bir puanın puanlar listesinde yer alıp almadığını keşfetmek. Arama, bilgisayar programlamasında yaygın bir görevdir. Aramaya adanmış birçok algoritma ve veri yapısı vardır.

```
public class LinearSearch {
  /** Listede bir anahtar bulma yöntemi */
  public static int linearSearch(int[] list, int key) {
    for (int i = 0; i < list.length; i++)
      if (key == list[i])
        return i;
    return -1;
  }
}
```



keyCompare anahtarı list[i] ile karşılaştırmak için i = 0, 1, …

Doğrusal Arama

Anahtar

eleman, anahtar, *sırasıyla* dizideki her eleman ile

dizi listesindeki. Yöntem, anahtar bir elemanla eşleşene kadar

bunu yapmaya devam eder veya

listede eşleşme bulunmadan liste tükenene kadar

devam eder. Eğer bir eşleşme yapılırsa, doğrusal arama

elemanın dizideki indeksini döndürür.

anahtara uyan. Eşleşme bulunamazsa,

arama sonuçları -1.

animasyon

Doğrusal Arama Animasyonu

Anahtar	Liste							
3	6	4	1	9	7	3	2	8
3	6	4	1	9	7	3	2	8
3	6	4	1	9	7	3	2	8
3	6	4	1	9	7	3	2	8
3	6	4	1	9	7	3	2	8
3	6	4	1	9	7	3	2	8

Fikirden Çözüme

```
/** Listede bir anahtar bulma yöntemi */
public static int linearSearch(int[] list, int key) {
  for (int i = 0; i < list.length; i++)
    if (key == list[i])
      return i;
  return -1;
}
```

Yöntemi izleyin

```
int[] list = {1, 4, 4, 2, 5, -3, 6, 2};
int i = linearSearch(list, 4);  // 1 döner
int j = linearSearch(list, -4); //  -1 döner
int k = linearSearch(list, -3); // 5 döner
```

Dizileri Sıralama

Sıralama, arama gibi, bilgisayar programlaması nda da yaygın bir görevdir. Birçok farklı algoritma Sıralama için geliştirilmiştir.

Arrays.sort Metodu

Sıralama, programlamada sıkça kullanıldığından, Java, int, double, char, short gibi bir diziyi sıralamak için birkaç aşırı yüklenmiş sıralama yöntemi sağlar. long ve float, java.util.Arrays sınıfında bulunmaktadır. Örneğin, aşağıdaki kod, bir dizi sayıyı ve bir dizi karakteri sıralar.

```
double[] numbers = {6.0, 4.4, 1.9, 2.9, 3.4, 3.5};
java.util.Arrays.sort(numbers);

char[] chars = {'a', 'A', '4', 'F', 'D', 'P'};
java.util.Arrays.sort(chars);
```